- Classes with a trivial default constructor, copy constructor, copy assignment operator, and destructor can be used in a union with a user-defined constructor and destructor.
- Classes with trivial copy assignment operators can be used with the `std::atomic<>` class template (see section 5.2.6) in order to provide a value of that type with atomic operations.

Just declaring the function as `= default` doesn't make it trivial—it will only be trivial if the class also supports all the other criteria for the corresponding function to be trivial—but explicitly writing the function in user code does *prevent* it from being trivial.

The second difference between classes with compiler-generated functions and user-supplied equivalents is that a class with no user-supplied constructors can be an *aggregate* and thus can be initialized with an aggregate initializer:

```
struct aggregate
{
    aggregate() = default;
    aggregate(aggregate const&) = default;

    int a;
    double b;
};
aggregate x={42,3.141};
```

In this case, `x.a` is initialized to `42` and `x.b` is initialized to `3.141`.

The third difference between a compiler-generated function and a user-supplied equivalent is quite esoteric and applies only to the default constructor and only to the default constructor of classes that meet certain criteria. Consider the following class:

```
struct X
{
    int a;
};
```

If you create an instance of class `X` without an initializer, the contained `int` (`a`) is *default initialized*. If the object has static storage duration, it's initialized to zero; otherwise, it has an indeterminate value that can